

C++: pointers & parametersL01b; W3

&x: address of x. *p: object at p.
p->m equals (*p).m; a[i] equals *(a+i). Pointer arithmetic advances by elements.

```
int x = 3;  
int* p = &x;  
int& r = x;    // alias; must initialize  
*p = 5;        // x and r now 5  
p = nullptr;   // changes p, not x
```

Parameter	Effect
T x	Copy; changing x does not change caller.
T* p	Copy of address; *p may change caller's object.
T& x	Alias; may change caller's object.
const T& x	No object copy; no mutation through x.

To change the caller's pointer itself, use T*& p (or a pointer to pointer). Arrays passed as T a[] act as pointers; pass length separately.

Allocation & lifetime

```
T* p = new T(...); delete p;  
T* a = new T[n]; delete[] a;  
p = nullptr; a = nullptr;
```

Local objects are destroyed at scope exit. Dynamically allocated objects live until deleted. Declaring T* p; creates a pointer, not a T object.

Watch: Never dereference null/uninitialized/dangling pointers. Do not use an object after deletion, double-delete, or delete a local stack object. Losing the last owning pointer leaks memory.

Classes & inheritanceL01b; L02a/d

Class: blueprint; **object:** instance. Encapsulation exposes an interface while hiding state.

public Accessible to clients.
private Class members and declared friends.
protected Also accessible to derived classes.

Default member access: **class** private; **struct** public. A derived class cannot directly access base private members merely by inheritance. **friend class** B; inside A grants B access to A's private/protected members. Friendship is not automatically mutual or transitive.

Construction, destruction, copying

C(...): constructor, no return type, overloadable; Can have **multiple**
~C(): destructor, no return type/arguments; Only **ONE** can exist.

C(const C& other): copy constructor. Implicit copying copies pointer values, not pointed-to nodes: owning lists need deep copying (and correct assignment/destruction) to avoid shared ownership errors.

Overloading vs overriding

Overload: same name, different parameter number/types; return type alone is insufficient.

Override: derived implementation of a virtual base method. Through a base pointer/reference, virtual calls select the actual object's method; nonvirtual calls use the static type. **Base::f()** explicitly calls the base method.

```
class Base {  
public:  
    virtual void f();  
    virtual ~Base() {}  
};  
class Child : public Base {  
public: void f() override;  
};
```

Templates & operatorsW4-5

```
template<class T>  
void mySwap(T& a, T& b) {  
    T t = a; a = b; b = t;  
}  
bool Food::operator>(const Food& f)  
{ return _cal > f._cal; }
```

a+b can call a.operator+(b). Returning a new object need not modify either input. Stream **operator<<** returns **ostream&** for chaining. Template definitions must be visible where instantiated (usually header/.hpp).

Linked-list costsL02a; W3-5

Singly linked node: item + **next**. Empty: **head==nullptr**; last node's **next==nullptr**. Doubly linked adds **prev**; circular links the last node back to the first.
Operation

	Array	SLL	DLL
Index access	$\Theta(1)$	$O(n)$	$O(n)$
Search (unsorted)	$O(n)$	$O(n)$	$O(n)$
Insert/remove head	$O(n)$	$\Theta(1)$	$\Theta(1)$
Append with tail*	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Remove tail	$\Theta(1)$	$O(n)$	$\Theta(1)$

*Array has spare capacity; lists maintain a tail pointer. DLL tail removal assumes a tail pointer. Array head deletion preserves order. SLL tail deletion still needs the predecessor. **Known location:** SLL insertion after a node or deletion after a predecessor is $\Theta(1)$. DLL deletion of a known node is $\Theta(1)$. Finding that location may cost $\Theta(n)$.

Safe head operations (size counter sz)

```
// push_head(x)  
Node* t = new Node(x);  
t->next = head; head = t;  
if (!tail) tail = t;  
++sz;  
  
// pop_head(): require head != nullptr  
Node* t = head;  
T x = t->item;  
head = head->next;  
if (!head) tail = nullptr;  
delete t; --sz;  
return x;
```

Destructor: repeatedly remove head, $\Theta(n)$. Handle empty/singleton cases; update head, tail and size consistently. Repeated head insertion reverses input order.

Reverse links: $\Theta(n)$ time, $\Theta(1)$ space

```
Node* prev = nullptr;  
Node* cur = head;  
tail = head;  
while (cur) {  
    Node* nxt = cur->next;  
    cur->next = prev;  
    prev = cur; cur = nxt;  
}  
head = prev;
```

extractMax: scan with current/predecessor and best/predecessor, unlink best, return item: $\Theta(n)$. Use strict > to keep the first equal maximum. Repeating extraction costs $\Theta(n^2)$; printing maxima gives descending order, prepending them gives ascending order.

ADTs & representationsL02d; W6

ADT specifies operations and behavior; a data structure implements them. Same ADT can have different costs with different representations. **Stack (LIFO):** push/pop at head, $\Theta(1)$.

Queue (FIFO): enqueue at tail, dequeue at head, $\Theta(1)$ with both pointers.

List: sequence with positional access/updates.

Dictionary: key-value insert, search, delete, predecessor/successor. **Set:** unique elements. **Set representations:** unsorted array membership $O(n)$; sorted array membership $O(\log n)$ but insertion/deletion $O(n)$. Merging two sorted sets supports union/intersection/difference in $O(n + m)$. Unsorted membership checks can take $O(nm)$. Unsorted append is constant only if spare capacity and no duplicate check is needed.

SearchingL03a

Linear search: best $\Theta(1)$, worst $\Theta(n)$. Binary search needs a **sorted random-access array**: worst $\Theta(\log n)$; iterative extra space $\Theta(1)$.

```
// search range [lo, hi)  
int lo = 0, hi = n;  
while (lo < hi) {  
    int m = lo + (hi-lo)/2;  
    if (A[m] == key) return m;  
    if (key < A[m]) hi = m;  
    else lo = m + 1;  
}  
return -1;
```

A linked-list midpoint needs linear traversal: halving its range does not imply logarithmic total time.

Asymptotic boundsL03a; TC notes

For sufficiently large $n \geq n_0$ and positive constants: $T \in O(f)$ Upper bound: $0 \leq T(n) \leq cf(n)$. $T \in \Omega(f)$ Lower bound: $0 \leq cf(n) \leq T(n)$. $T \in \Theta(f)$ Tight: $c_1 f(n) \leq T(n) \leq c_2 f(n)$. $f \in O(g) \iff g \in \Omega(f)$. A linear function is also $O(n^2)$, but not $\Theta(n^2)$. **Bound type and input case are separate:** O does not mean “worst case” and Ω does not mean “best case”.

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!$$

Drop fixed factors/lower terms **after counting**. Log bases differ by a constant: $\log_b n = \log_a n / \log_a b$.

Combine actual costs

Sequential blocks: add; independent nested work: multiply; dependent bounds: **sum per iteration**. A worst-case branch bound includes condition cost plus the most expensive feasible branch. Check termination first.

Useful sums

$$\sum_{i=1}^n 1 = n; \quad \sum_{i=1}^n i = \frac{n(n+1)}{2}.$$
$$\sum_{i=0}^k r^i = \frac{r^{k+1}-1}{r-1} \quad (r \neq 1).$$
$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \Theta(n); \quad \sum_{i \geq 0} n/2^i = \Theta(n).$$
$$\sum_{i=1}^n \frac{1}{i} = \Theta(\log n).$$

Loop patterns & trapsW5-6

Assume constant-time bodies and positive n .

Pattern	Tight time
i=0; i<n; i+=100	$\Theta(n)$
i=1; i<n; i*=2	$\Theta(\log n)$
i=0; i<n*n; i+=2*n	$\Theta(n)$
i=1; i<n; i+=n/10+1	$\Theta(1)$
Outer n; inner n/2	$\Theta(n^2)$
Outer i = 1, ..., n; inner i	$\Theta(n^2)$
Outer doubles; inner runs n	$\Theta(n \log n)$
Outer doubles; inner runs i	$\Theta(n)$
Outer doubles; inner steps by i to n	$\Theta(n)$
Outer i = 1, ..., n; inner steps by i to n	$\Theta(n \log n)$

Watch: j=0; j<n; j*=2 never advances for $n > 0$. Uninitialized indices give undefined behavior.

Changing bound: start $k = n$, repeatedly halve k , then run k inner iterations. Total inner work $n/2 + n/4 + \dots = \Theta(n)$, even if the outer loop executes n times (include its $\Theta(n)$ overhead). **Why not multiply loose bounds?**

For $i = 1, 2, 4, \dots < n$, each inner cost n/i : $\theta(n)$, see **Useful sums** formula for $n/2^i$. Calling every inner loop $O(n)$ gives a valid but loose $O(n \log n)$ bound. **W6 triple loop:** i doubles, j steps by i , and $k = 0, \dots, j - 1$. For fixed i , work is $\Theta(n^2/i)$; summing geometric terms gives $\Theta(n^2)$.

Recursion: Master TheoremL03-05; W6

$$T(n) = aT(n/b) + \Theta(n^d)$$

Shrink	Result
Divide ($n/2, n/3$)	$\log n$ involved
Subtract ($n - 1, n - 2$)	exponential or factorial
Branches	Result (Big-O)
1 + divide	$\log n$
2 + divide	$n \log n$ or n
2 + subtract	2^n
k + subtract	$k^n (k \geq 2, k = 1 \vdash n)$
Case	Result
$a < b^d$	$O(n^d)$
$a = b^d$	$O(n^d \log n)$
$a > b^d$	$O(n^{\log_b a})$

Recurrence	Time
$T(n) = T(n - c) + \Theta(1)$	$\Theta(n)$
$T(n) = T(n - c) + \Theta(n)$	$\Theta(n^2)$
$T(n) = T(n/b) + \Theta(1)$	$\Theta(\log n)$
$T(n) = T(n/b) + \Theta(n)$	$\Theta(n)$
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
$T(n) = 10T(n/10) + \Theta(n)$	$\Theta(n \log n)$
$T(n) = 3T(n/5) + \Theta(1)$	$\Theta(n^{\log_5 3})$
$T(n) = 2T(n - 1) + \Theta(1)$	$\Theta(2^n)$

Use the exact algorithm variant in the question. A completed-pass invariant may not hold halfway through a swap, shift, or partition.

Sorting comparisonL04 BSIM; L04b; L05

All entries below are $\Theta(\cdot)$; average assumes a random order, or random pivots for QuickSort. QuickSort row assumes distinct keys and ordinary two-way recursion.

Algorithm	Best	Average	Worst
Bubble (early exit)	n	n^2	n^2
Selection	n^2	n^2	n^2
Insertion	n	n^2	n^2
Merge	$n \log n$	$n \log n$	$n \log n$
Quick	$n \log n$	$n \log n$	n^2

Algorithm	Extra space	Stable?
Bubble / Insertion	$\Theta(1)$	Yes
Selection (swap)	$\Theta(1)$	No
Merge (array)	$\Theta(n)$	Yes*
Quick (expected)	$O(\log n)$	No
Quick (worst)	$O(n)$	No

*Merge must take from the left on ties. Bubble/Insertion use strict $>$ for swaps/shifts. QuickSort partition itself uses $\Theta(1)$ extra space; recursion adds stack space. Fixed-pass BubbleSort has $\Theta(n^2)$ best time too. **Stable**: equal-key records retain relative input order. Example: $[2_a, 2_b, 1]$ becomes $[1, 2_b, 2_a]$ after selection's first swap: unstable.

Sorting invariantsDetective; W5

Let A^0 be the original input. After k completed passes in the usual ascending versions:

Bubble: fixed largest suffix/last k fixed

Left-to-right **adjacent comparisons** move a maximum to the end. Last k entries are the k largest, in final positions. A small value moves left at most one place per pass. (**scramble head**)

Selection: fixed smallest prefix/first k fixed

First k entries are the k smallest overall, sorted and already in final positions. Remaining entries may be rearranged by long-distance swaps. (**scramble tail**)

Insertion: sorted original prefix

First $k + 1$ entries are a sorted permutation of $A^0[0..k]$. Suffix $A[k + 1..n - 1]$ is unchanged. Prefix elements need **not** be the smallest overall or in final positions.

Watch: Both selection and insertion have sorted prefixes. Check *which elements* the prefix contains and whether the untouched suffix matches the original.

Merge: sorted blocks

Completed merges create sorted runs within their original subarray boundaries. Bottom-up run lengths double; top-down recursion may leave mixed run sizes. No global rank prefix/suffix is required.

Quick: pivot-separated regions

After partition, everything to a pivot's left is $\leq$ it and everything to its right is $\geq$ it. Pivot is in a valid final position, though regions may remain unsorted. Being in the correct index alone does not prove an element was a pivot. **Worksheet key**: A = Insertion; B = Selection; C = Bubble; D = Merge; E = Quick (first pivot Carol).

Insertion & selection kernelsL04 BSIM

```
// insertion: prefix grows by one
for (int j=1; j<n; ++j) {
    auto key=A[j]; int i=j-1;
    while (i>0 && A[i]>key) {
        A[i+1]=A[i]; --i;
    }
    A[i+1]=key;
}

// selection: choose suffix minimum
for (int j=0; j<n-1; ++j) {
    int k=j;
    for (int i=j+1; i<n; ++i)
        if (A[i]<A[k]) k=i;
    swap(A[j], A[k]);
}
```

Selection: $n(n - 1)/2$ key comparisons; at most $n - 1$ nontrivial swaps. Insertion shifts once per inversion (a pair $i < j$ with $A[i] > A[j]$); efficient for nearly sorted/small inputs.

MergeSortL04 BSIM; W6

Divide into halves; recursively sort each; merge using two pointers: take the smaller front, then append leftovers. Take **left on equality** for stability. Merge lengths a, b : $\Theta(a + b)$ time, at most $a + b - 1$ key comparisons if both nonempty.

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

There are $\Theta(\log n)$ levels and $\Theta(n)$ work per full level. Reuse/free temporary arrays: peak space $\Theta(n)$, not the $\Theta(n \log n)$ cumulative allocation/work. **Linked-list MergeSort**: find middle with slow/fast pointers, split links, sort both, merge by relinking. Still $\Theta(n \log n)$ time; $O(1)$ merge workspace with iterative merge, plus $O(\log n)$ recursive call stack.

QuickSort & partitionL04b QSort

Choose pivot; partition; recursively sort strict left/right parts; no merge needed. Base case: subarray size at most one, including empty. **Lecture partition, distinct keys**: first pivot; scan low rightwards past smaller values and high leftwards past larger ones; swap misplaced items; finally move pivot to boundary. One partition is $\Theta(n)$. **Duplicates**: strict scans may stop on equal values and repeatedly swap without advancing. Ensure progress; grouping all equals also avoids recurring on them.

Three-way variant (inclusive range)

This compact implementation groups $< v, = v, > v$. Its exact intermediate array can differ from the lecture's two-pointer/pack-duplicates version.

```
void quick(A, int l, int r) { // pseudo-C++
    if (l >= r) return;
    auto v=A[l]; // copy pivot value
    int lt=l, i=l+1, gt=r;
    while (i <= gt) {
        if (A[i] < v) {
            swap(A[lt], A[i]); ++lt; ++i;
        } else if (A[i] > v) {
            swap(A[i], A[gt]); --gt;
        } else ++i;
    }
    quick(A, l, lt-1);
    quick(A, gt+1, r);
}
```

Loop regions: $A[l..lt - 1] < v$; $A[lt..i - 1] = v$; $A[i..gt]$ unknown; $A[gt + 1..r] > v$. After swapping with **gt**, do not increment **i**: the incoming item is unexamined. All-equal input takes $\Theta(n)$ with three-way grouping.

QuickSort analysisL05; W6

For distinct keys with pivot rank $p + 1$:

$$T(n) = T(p) + T(n - p - 1) + \Theta(n).$$

Balanced: $2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Worst: pivot repeatedly min/max: $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$. Sorted/reversed input is bad for first/last pivot. A middle-index pivot is not necessarily the median. **Constant-fraction split**: even $1 : 9$ is enough: $T(n) = T(n/10) + T(9n/10) + \Theta(n) = \Theta(n \log n)$. Longest path obeys $n(9/10)^h \approx 1$, hence $h = \Theta(\log n)$. **Random pivot**: expected $\Theta(n \log n)$ for each distinct-key input, but worst $\Theta(n^2)$ remains. Randomized analysis averages algorithm choices; average-case analysis averages an input distribution. **Paranoid QuickSort**: retry until pivot rank lies in the middle 80% (roughly). Good-pivot probability $p \approx 0.8$; expected trials $1/p \approx 1.25$; expected partition work $\Theta(n)$.

Watch: Each side needs a constant *fraction* of n , not merely a constant number of elements. Expected trials are not a guaranteed maximum.

For independent trials with fixed success probability p : expected successes in m trials = mp ; expected trials until first success = $1/p$.

Selection & matching applicationsw6

QuickSelect: partition, then recurse only into the side containing target index $k - 1$ for the k th smallest. Expected $\Theta(n)$; worst $\Theta(n^2)$. Balanced recurrence: $T(n) = T(n/2) + \Theta(n)$. Skip equal block in a three-way implementation. **Children/shoes**: random child partitions shoes; matching shoe partitions children; recurse on corresponding smaller/larger groups. Expected $O(n \log n)$, worst $O(n^2)$, assuming distinct matching sizes.

Trees & BSTsL05c Tree Def/Traversal

Root: no parent. **Leaf**: no children. **Siblings**: same parent. **Subtree**: a node and all descendants. Binary tree: each node has at most two children (left/right); either subtree may be empty. **Depth**: edges from root; root depth 0.

Height: longest downward path in edges:

$$h(\emptyset) = -1, \quad h(v) = 1 + \max(h(v_L), h(v_R)).$$

Leaf height 0. A skewed tree has height $n - 1$; a balanced tree has height $O(\log n)$. Computing height from scratch visits the whole subtree: $\Theta(n)$.

BST ordering is global within subtrees

All keys in the left subtree $< v.key <$ all keys in the right subtree (distinct-key convention). Checking only immediate children is insufficient. Insertion order determines shape; identical keys can yield different shapes. **Search/insert**: compare at root, go left for smaller, right for larger; insert at the missing child position. Equality follows the chosen duplicate policy. **Min/max**: follow left/right links respectively.

Cost: search, insert, min/max, successor/predecessor take $O(h + 1)$: balanced $O(\log n)$, skewed $O(n)$. An ordinary BST is **not automatically balanced**.

Successor / predecessor

Successor of key x is the smallest key strictly $> x$. If a present node has a right subtree, use its minimum; otherwise use the lowest ancestor whose left subtree contains that node.

```
// works even when x is absent
Node* ans=nullptr;
Node* cur=root;
while (cur) {
    if (cur->key > x) {
        ans=cur; cur=cur->left;
    } else cur=cur->right;
}
return ans; // nullptr if no successor
```

Predecessor is symmetric: largest key $< x$; left-subtree maximum, or last candidate when moving right.

Traversal ordersL05c

Every recursive traversal starts: if node is null, return. Let N = visit node, L/R = traverse subtree.

Traversal Order Use / property

Preorder	N L R	Root appears first
Inorder	L N R	BST keys ascending (sorted)
Postorder	L R N	Children before parent
Level order	BFS	Queue, left then right

All **traversals**/visit n nodes in $\Theta(n)$ time. DFS recursion uses $O(h + 1)$ space. Level order uses $O(w)$ queue space, where w is maximum width.

Peak findingL03a; TC notes

1D peak: $A[i] \geq A[i - 1], A[i + 1]$; use virtual boundaries $A[-1] = A[n] = -\infty$. Check middle; if not a peak, recurse toward a *larger neighbor*. Either larger side contains a peak. Worst $O(\log n)$; array need not be sorted. **Global maximum**: unsorted $\Theta(n)$; sorted ascending $\Theta(1)$ by returning last element. **2D, n rows and m columns**: compare direct up/down/left/right neighbors. Use non-strict peaks, or distinct values for strict peaks. Form column global maxima and seek a 1D peak among them. Compute all columns first: $O(nm)$. Evaluate only columns examined by binary peak finding: $O(n \log m)$ for $m \geq 2$ ($O(n)$ when $m = 1$). Arbitrary *local* column maxima do not suffice.

Two-stack sorting patternW6 Quiz 1

Head is stack top; **print()** traverses head to tail. To make L2 ascending:

```
while (!L1.empty()) {
    int x=L1.pop_head();
    while (!L2.empty() && L2.head(<x)
        L1.push_head(L2.pop_head());
        L2.push_head(x);
}
```

Move smaller items aside before prepending a larger item. L2 stays ascending from head to tail. Worst $\Theta(n^2)$ time; only constant temporary storage beyond the two lists.